

Remove `try_append_range` from `inplace_vector` for now

Document #: P4022R0 [[Latest](#)] [[Status](#)]
Date: 2026-02-22
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Jonathan Wakely
<cxx@kayari.org>
Tomasz Kamiński
<tomaszkam@gmail.com>

Contents

[1 Introduction](#)

[2 Issues](#)

[2.1 Semantics](#)

[2.2 Return Type](#)

[3 Proposal](#)

[4 References](#)

§ 1 Introduction

In [\[P3981R0\]](#) ([Better return types in `std::inplace\_vector` and `std::exception\_ptr` cast](#)), one of the changes proposed in that paper was changing the return type of `try_append_range`:

```
template<container-compatible-range<T> R>  
- constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);  
+ constexpr ranges::borrowed_subrange_t<R> try_append_range(R&& rg);
```

During the discussion of this paper at a recent [LEWG telecon](#), a few issues came up with this particular member function that lead us to conclude that we should remove it for C++26 so that we have more time to figure out how it should behave in C++29. This also addresses [PL-006](#).

§ 2 Issues

There are two issues with `try_append_range`:

1. What should this function even *do*?

2. What should this function return?

§ 2.1 Semantics

There are currently three functions in `std::inplace_vector<T, N>` whose name starts with `try_`:

1. `try_emplace_back`
2. `try_push_back` (two overloads)
3. `try_append_range`

The first two are simply fallible versions of the corresponding non-prefixed member functions, whose semantics are straightforward and obvious, which is why we also argued in [\[P3981R0\]](#) that they should return `std::optional<T&>`. They either succeed (returning what the other function returns) or they fail (returning nothing).

But `try_append_range` is different — it’s not simply a case of success or failure.

If we attempt to add one element to an `inplace_vector`, there are only two options: it either fits, or it didn’t. But if we attempt to attempt to add `N` elements, there are multiple options: they all fit, none of them fit, or some of them fit. What do we want to happen in this case?

The current specified behavior is:

16 *Effects:* Appends copies of initial elements in `rg` before `end()`, until all elements are inserted or `size() == capacity()` is `true`. Each iterator in the range `rg` is dereferenced at most once.

An alternative formulation would be to try to check to see if all of the elements in `rg` can fit — and fail if they all can’t. This could be a cheap check for a `sized_range`, for instance.

Which semantics do we want?

If we allow for partial insertion (as the current specification does), is that really a “failure”? That suggests that `try_append_range` may not even be the right name for the operation? Perhaps something like `append_some` is better for this particular semantic?

§ 2.2 Return Type

The current specification is to return an iterator pointing to the first non-inserted element of `rg`. But [\[P3981R0\]](#) points out that this is clunky, and suggests instead that we return the whole subrange of non-inserted elements.

This is more useful, but has a big problem: `subrange` has an explicit conversion to `bool`, such that a non-empty subrange is truthy. This is problematic in this case:

```

if (v.try_push_back(elem)) {
    // we successfully inserted elem into v
}

if (not v.try_append_range(rg)) {
    // we successfully inserted all of rg into v
}

if (v.try_append_range(rg)) {
    // there is at least 1 element of rg that we did
    // not insert into v
}

```

The API inconsistency is not great, and inventing a new subrange-but-non-truthy type seems like a poor answer.

Additionally, with regards to the question of [semantics](#), there is a question of whether any kind of “truthy” return type is misleading.

For a fallible try_append_range that would be an all-or-nothing analogue to append_range, that doesn’t try to conditionally insert elements, returning `bool` is sensible. But for a function with the current semantics where append_some might be a better name, you might simply want to return all of the information in a way that isn’t mis-usable. Perhaps something like:

```

template <class R>
struct append_some_return {
    // subrange into the elements inserted into the inplace_vector
    subrange<iterator> inserted;

    // subrange into the remaining elements that could not be inserted
    borrowed_subrange_t<R> remaining;
};

template <container-compatible-range<T> R>
constexpr append_some_return<R> append_some(R&& rg);

```

Something like this shape gives us nice ergonomics for dealing with the result, without any confusing/misleading conversions to `bool`.

But this is a fairly substantive API change.

§ 3 Proposal

We feel that there are still quite some design discussions to have about this particular member function. So let’s just have them for C++29.

Remove try_append_range from 23.3.16.1 [\[inplace.vector.overview\]](#):

```

namespace std {
    template<class T, size_t N>
    class inplace_vector {
    public:

        // ...

        template<class... Args>
            constexpr pointer try_emplace_back(Args&&... args);
        constexpr pointer try_push_back(const T& x);
        constexpr pointer try_push_back(T&& x);
-     template<container-compatible-range<T> R>
-         constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);

        // ...

    };
}

```

And from 23.3.16.5 [\[inplace.vector.modifiers\]](#):

```

template<container-compatible-range<T> R>
constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);

```

15 ~~*Preconditions:* `value_type` is `Cpp17EmplaceConstructible` into `inplace_vector` from `*ranges::begin(rg)`.~~

16 ~~*Effects:* Appends copies of initial elements in `rg` before `end()`, until all elements are inserted or `size()` == `capacity()` is `true`. Each iterator in the range `rg` is dereferenced at most once.~~

17 ~~*Returns:* The first iterator in the range `ranges::begin(rg) + [0, n)` that was not inserted into `*this`, where `n` is the number of elements in `rg`.~~

18 ~~*Complexity:* Linear in the number of elements inserted.~~

19 ~~*Remarks:* Let `n` be the value of `size()` prior to this call. If an exception is thrown after the insertion of `k` elements, then `size()` equals `n+k`, elements in the range `begin() + [0, n)` are not modified, and elements in the range `begin() + [n, n+k)` correspond to the inserted elements.~~

And bump the feature-test macro in 17.3.2 [\[version.syn\]](#):

```

- #define __cpp_lib_constexpr_inplace_vector      202502L // also in
  <inplace_vector>
+ #define __cpp_lib_constexpr_inplace_vector      2026XXL // also in
  <inplace_vector>

```

§ 4 References

[P3981R0] Barry Revzin, Jonathan Wakely, and Tomasz Kamiński. 2026-01-27. Better return types in `std::inplace_vector` and `std::exception_ptr_cast`.

<https://wg21.link/p3980r0>